Féidearthachtaí as Cuimse
Infinite Possibilities

AI: Theory to Practice

Modelling & Evaluation Report

Group 6:
Max Sri-Uttha X00213723
Odhran Chamberlain X00217322
Oleh Kravets X00219607

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Plagiarism declaration:

We confirm that this report is our own work. All external sources, tools, and assistance (including GenAI) have been appropriately acknowledged.

Contribution statement:

Max Sri-Uttha X00213723 – 33 % : Built and reported Random Forest, Linear Support Vector Machine, and LightGBM. Contributed to the deep learning models, especially TabNet, and led the evaluation outputs such as cross-validation summaries, plots, and the final comparison table.

Odhran Chamberlain X00217322 – 33 % :Built and reported Decision Tree, Naive Bayes, and XGBoost. Contributed to the deep learning models, especially the Multilayer Perceptron models, and helped with the feature engineering and preprocessing write-up.

Oleh Kravets X00219607 – 33 % : Built and reported Logistic Regression, k-Nearest Neighbours, Histogram Gradient Boosting, and CatBoost. Contributed to the deep learning models, especially Residual MLP and FT-Transformer, and helped with the final model comparison content.

Dataset source: https://huggingface.co/datasets/mstz/nbfi

Dataset license: CC0: Public Domain

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

0. Problem, Dataset & Preprocessing Overview

In this project we solve a **supervised machine learning** problem using **binary classification**. The aim is to **predict loan default** by estimating whether a customer has defaulted based on their application and background features.

The dataset used is the **NBFI Vehicle Loan Repayment Dataset** from Hugging Face. It is provided as a **CSV file** with **1.59 MB** size, **8,308 rows**, and **36 columns** (**35 features** plus **1 target**). Examples of key features include income, credit, loan_annuity, income_type, education_level, city_rating, weekday_of_application, and hour_of_application.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

0. Problem, Dataset & Preprocessing Overview

From **EDA**, the main findings were: the target is **highly imbalanced** with only about ~7% defaults. Financial variables like income, credit, and loan_annuity are strongly **right-skewed** with **extreme high values**. Score_2 contains a suspicious **off-scale value** (100) while most values are in [0,1]. We also found **high redundancy** between nr_children and nr_family_members (correlation about 0.9).

To make the dataset model-ready, we applied a **preprocessing pipeline**: removed **437 duplicate rows**, **dropped constant / zero-variance columns**, fixed **incorrect data types**, **imputed missing** with "Unknown", handled extremes using **capping** and **log transforms** on money features, corrected score_2 by **clipping** the off-scale value, **reduced multicollinearity** by dropping, and converted features to numeric form using **0/1 encoding**, **one-hot encoding**, and **min-max scaling** for scale-sensitive models. The final outputs are **X** and **y** for modelling.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

1. Feature Engineering and Feature Selection

We focused on simple feature engineering to make inputs easier for models. **After the CA1 Q&A**, we replaced the cyclical hour encoding (hour_sin, hour_cos) with five clear time-of-day categories: before bank open, before noon, noon break time, after noon, and after bank close. We also encoded categoricals into numbers: binary flags to 0 or 1, one-hot for nominal features (including "Unknown" income_type), and kept ordinal features like education_level and city_rating as ordered integers.

We also applied transformations to reduce the impact of extreme values and heavy skew. Financial variables had large right tails, so we used a cap plus log-style transform approach on money features, and we fixed the broken scaling issue in score_2 by clipping the off-scale value. These steps help prevent a small number of very large values from dominating training while keeping high-exposure cases in the dataset.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

1. Feature Engineering and Feature Selection

Feature selection was done using EDA checks. First, we removed constant (zero-variance) columns because they cannot help separate default from non-default. Second, we used correlation analysis to reduce redundancy: nr_children and nr_family_members were almost the same signal (correlation around 0.9), so we dropped nr_children and kept nr_family_members. We noticed credit and loan_annuity were also correlated, but we kept both because they represent different business meaning (loan size versus repayment obligation).

Finally, we applied feature scaling for models that are sensitive to feature ranges. We created a min-max scaled version of the continuous columns (with an _mm suffix) to support distance-based models. We did not scale binary or one-hot encoded columns, since they are already on a consistent 0 or 1 scale.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

2. Modelling

In this section we follow a next modelling workflow:

- Start from the final CA1 model-ready datasets: X (features) and y (target).
- Split the data using 60/20/20 split: train, validation, and test. Fixed random state used for repeatability.
- Choose evaluation metrics suited to class imbalance: PR AUC as the main metric, plus Balanced Accuracy and Macro F1. We keep confusion matrix for error analysis.
- For each model, train on the training set, then tune key hyperparameters using the validation set.
- After choosing the best settings, run 5-fold stratified cross-validation on train plus validation to get a more stable estimate.
- Retrain the final version of the model on train plus validation, then evaluate once on the untouched test set.
- Record all results in one shared results table for comparison and later discussion.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

2. Modelling

Although the assignment requires at least four supervised models, we evaluated a broader set because this is a difficult imbalanced classification task. Loan repayment is also influenced by human behaviour, which is not fully captured by application features. There is also a data quality risk because some applicants may report details inaccurately or strategically, which adds noise and reduces predictability. For these reasons, we tested a wider range of algorithms to find a model that is more robust to imbalance, noise, and non-linear patterns. Models we used (15 total):

Machine Learning		Deep learning
Logistic Regression	Linear SVM	MLP
Decision Tree	Histogram Gradient Boost	MLP with Dropout
Random Forest	XGBoost	Residual MLP
k-Nearest Neighbours	LightGBM	TabNet
Naive Bayes	CatBoost	FT-Transformer

Model 1: Logistic Regression

Rationale. We use Logistic Regression as our first baseline model for predicting default. It is simple, fast, and gives us a clear starting point before trying more complex methods on this imbalanced dataset. It works well as a baseline for binary classification and produces probabilities, which is useful for precision–recall evaluation.

Advantages. Quick to train and easy to reproduce. More interpretable than tree ensembles and deep learning. Regularization helps reduce overfitting when we have many encoded features. We can handle class imbalance directly using balanced class weights.

Limitations. It is a linear model, so it can miss non-linear patterns and feature interactions. It may underfit when default behavior is complex and noisy.

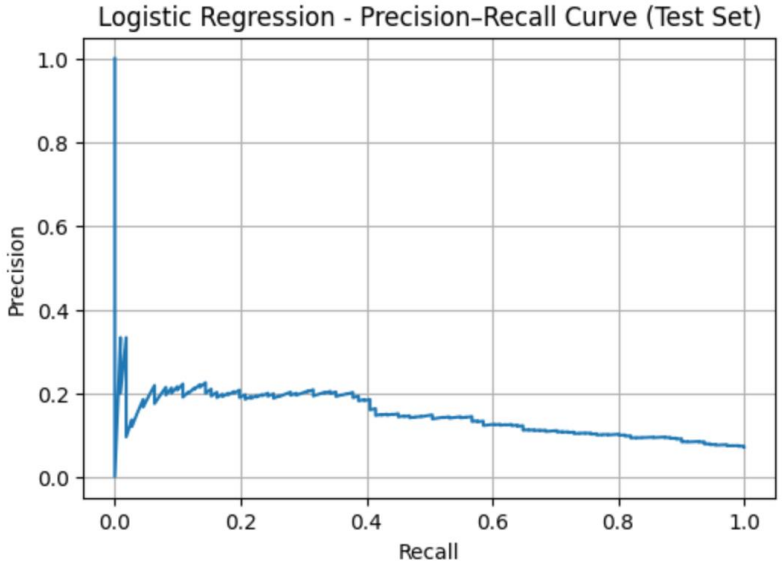
Hyperparameter. We fixed solver = "liblinear" and max_iter = 5000 to ensure stable training.

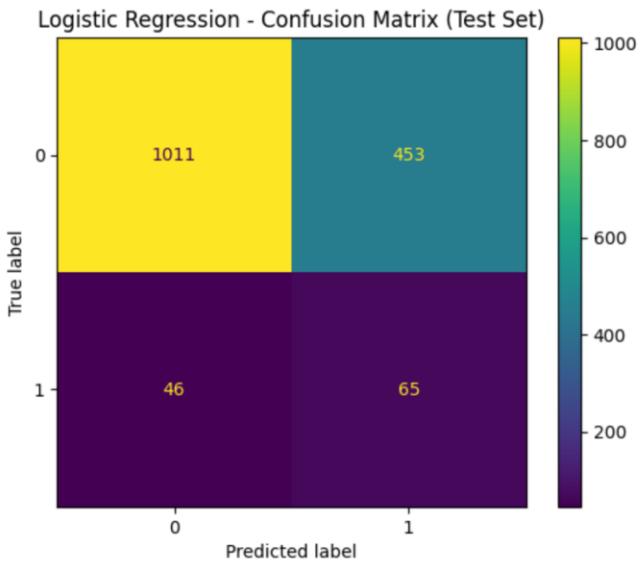
Imbalance. We used class_weight = "balanced" to address class imbalance.

Overfitting. We tuned the regularization strength C using a small grid: 0.01, 0.1, 1.0, 10.0, 100.0. We selected the best C using the validation average precision score, then evaluated once on the test set. We also ran 5-fold stratified cross-validation on the combined train and validation data to report a more stable performance estimate.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 1: Logistic Regression





Metric	Mean	Std
PR AUC	0.220787	0.034159
Balanced Accuracy	0.684099	0.011757
Macro F1	0.530194	0.005931

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 2: Decision Tree

Rationale. We use a Decision Tree because it can learn non-linear rules and simple if-then patterns. It is a good step after Logistic Regression, especially for tabular data.

Advantages. It can capture feature interactions, it does not need feature scaling, and it is easy to explain at a high level.

Limitations. A single tree can overfit easily, especially with many features and noisy behaviour in the target.

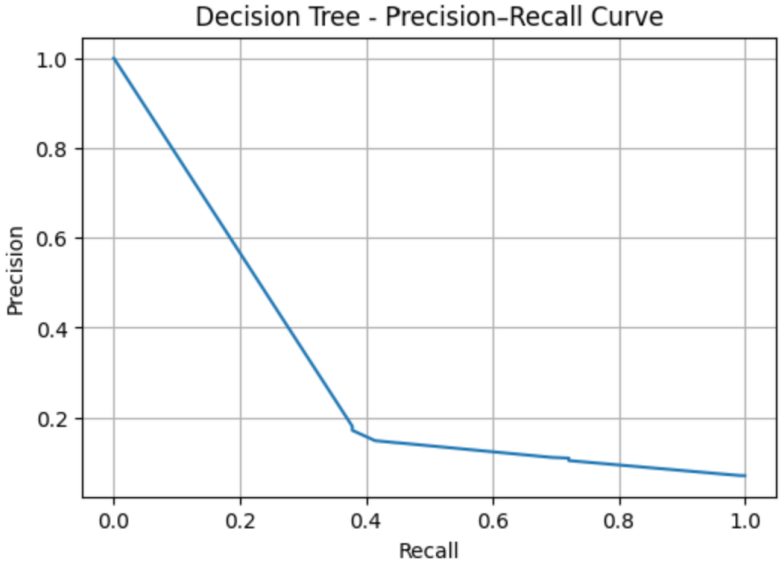
Hyperparameter. We tuned the main complexity settings on the validation set: max_depth, min_samples_split, and min_samples_leaf. We selected the best combination using validation precision–recall area under the curve.

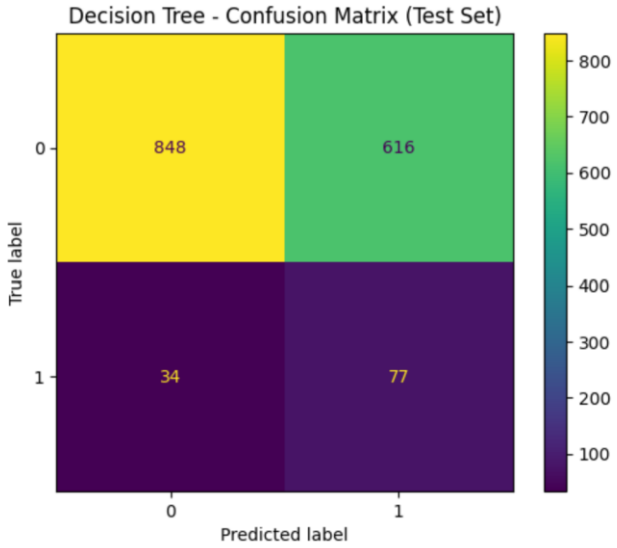
Imbalance. We used class_weight = “balanced” so the model pays more attention to the default class.

Overfitting. We controlled overfitting by limiting tree complexity through the tuned parameters: max_depth (tested: 3, 5, 7, 10, None), min_samples_split (tested: 2, 5, 10), min_samples_leaf (tested: 1, 2, 5).

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 2: Decision Tree





Metric	Mean	Std
PR AUC	0.137858	0.008787
Balanced Accuracy	0.667937	0.010841
Macro F1	0.457918	0.015391

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 3: Random Forest

Rationale. We use Random Forest because it is a strong model for tabular data and it reduces the instability of a single Decision Tree by averaging many trees.

Advantages. It captures non-linear patterns and feature interactions, it usually generalises better than one tree, and it does not require feature scaling.

Limitations. It is less interpretable than Logistic Regression, can be slower to train, and it can still overfit if trees are too complex.

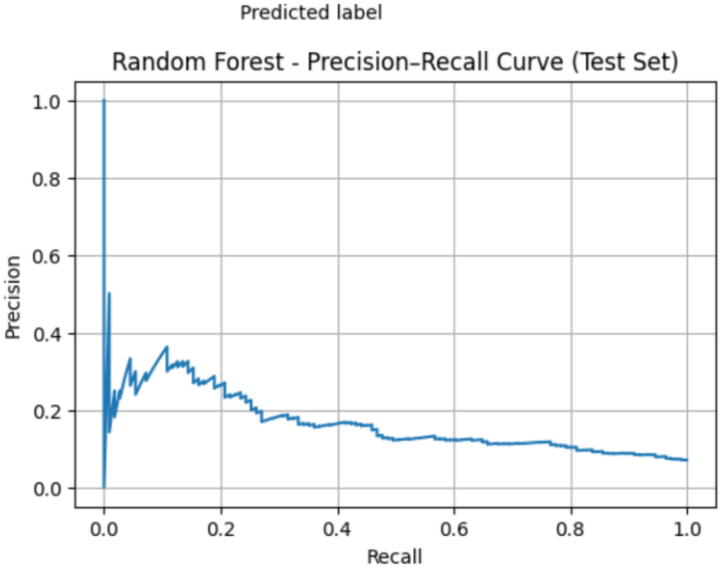
Hyperparameter. In our notebook we tuned a small grid: n_estimators = 200, max_depth in [None, 10], min_samples_split in [2, 10], min_samples_leaf in [1, 5], and max_features = "sqrt". We selected the best setting using validation PR AUC.

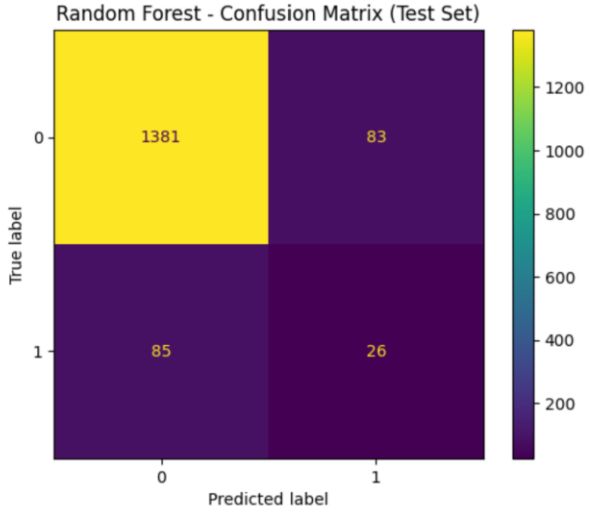
Imbalance. We used class_weight = "balanced_subsample" so each tree gives more importance to the minority class during training.

Overfitting. We limited tree complexity using max_depth, min_samples_split, and min_samples_leaf, then checked stability with 5-fold stratified cross-validation on the combined train and validation data before reporting final performance on the test set.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 3: Random Forest





Metric	Mean	Std
PR AUC	0.212925	0.025418
Balanced Accuracy	0.587197	0.014431
Macro F1	0.595913	0.014826

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 4: k-Nearest Neighbors

Rationale. We use k-Nearest Neighbors because it is a simple distance-based baseline that can work well on scaled tabular data, and it gives us a useful comparison against tree models.

Advantages. It is easy to understand, it can model non-linear boundaries, and it does not assume a linear relationship between features and default.

Limitations. It can struggle with many features and noisy data, it can be slow at prediction time, and it is very sensitive to feature scaling and to the choice of k.

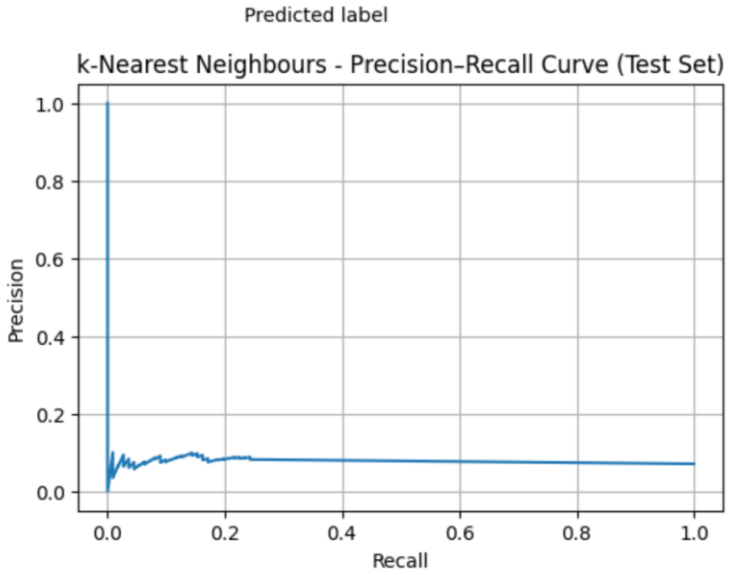
Hyperparameter. In our notebook we tuned a small grid: n_neighbors = 3, 5, 7, 11, 15; weights = uniform or distance; p = 1 or 2. We selected the best setting using validation PR AUC (average precision) and then confirmed stability with 5-fold stratified cross-validation.

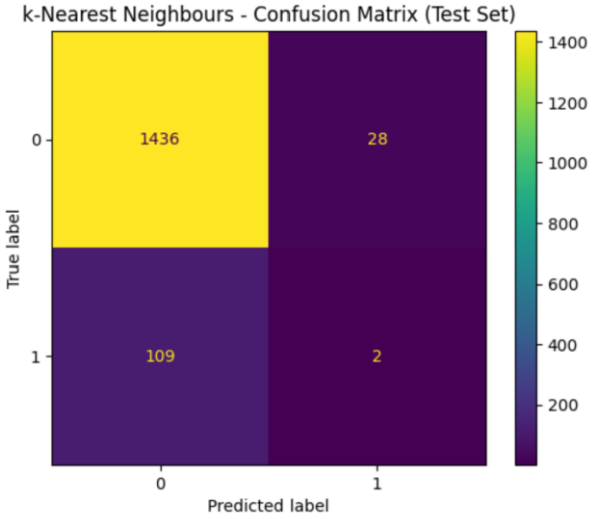
Imbalance. k-Nearest Neighbours does not support class weights, so we rely on stratified splitting and PR AUC-based evaluation to judge performance fairly on the rare default class.

Overfitting. We reduce overfitting mainly by tuning n_neighbors (small k can overfit, larger k smooths the decision boundary) and by comparing validation and cross-validation results before reporting final performance on the test set.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 4: k-Nearest Neighbors





Metric	Mean	Std
PR AUC	0.099428	0.010298
Balanced Accuracy	0.517272	0.011844
Macro F1	0.517582	0.018808

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 5: Naive Bayes

Rationale. We use Gaussian Naive Bayes as a very fast baseline. It is simple and gives us a quick reference point before moving to more complex models.

Advantages. It trains very quickly, works fine with many features, and gives probability outputs that we can evaluate using precision–recall metrics.

Limitations. It makes strong independence assumptions, so it often underfits on real credit data where features interact and patterns are not linear.

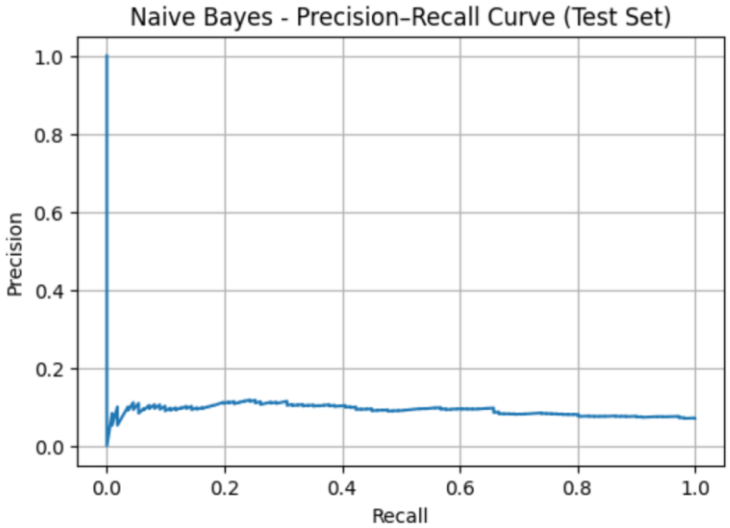
Hyperparameter. We tuned var_smoothing using this grid: 1e-12, 1e-11, 1e-10, 1e-9, 1e-8. We selected the best value using the validation average precision score.

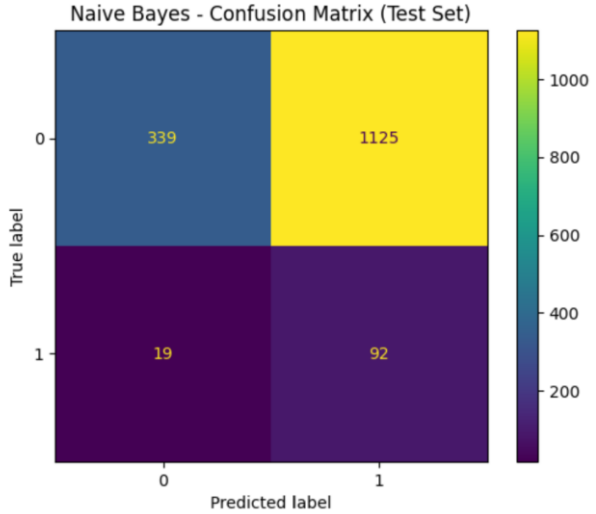
Imbalance. Naive Bayes does not support class weights, so we rely on stratified splitting and PR-based evaluation. In the notebook we used test_size = 0.20, then split the remaining training data with test_size = 0.25 for validation, both with stratification and random_state = 42.

Overfitting. Overfitting is usually not the main issue for Naive Bayes. Here we mainly stabilise the model using var_smoothing, and we check generalisation using 5-fold StratifiedKFold cross-validation before reporting final test set results.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 5: Naive Bayes





Metric	Mean	Std
PR AUC	0.124086	0.009220
Balanced Accuracy	0.531463	0.021165
Macro F1	0.173803	0.044846

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 6: Linear SVM

Rationale. We use a Linear SVM because it often works well on high-dimensional tabular data, especially after one-hot encoding. It is also a good comparison to Logistic Regression because both are linear, but SVM can behave differently with the same features.

Advantages. It can perform well with many features, it does not need complex feature engineering, and it is usually more robust than a single tree. It also gives a clear decision score that we can use for precision–recall evaluation.

Limitations. It is sensitive to feature scaling and to the choice of regularisation strength. It can also be slower than simpler baselines, and it can miss non-linear patterns unless we use a non-linear kernel.

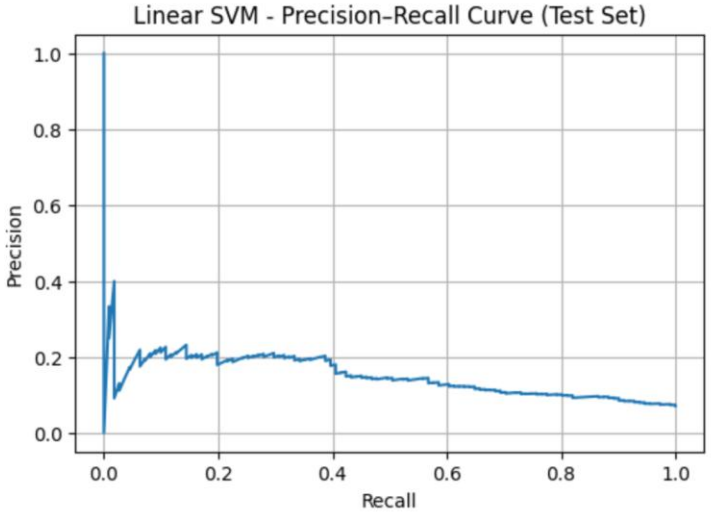
Hyperparameter. In the notebook we tuned only the regularisation strength C using a small grid: 0.01, 0.1, 1.0, 10.0. We kept dual = "auto" and max_iter = 20000 fixed for stable training.

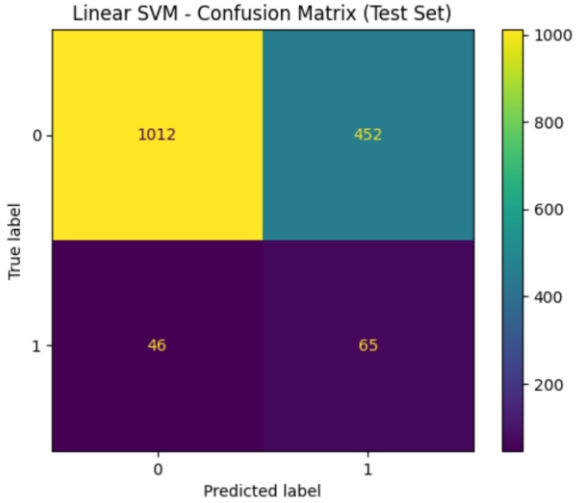
Imbalance. We handled class imbalance using class_weight = "balanced", so the minority default class has higher importance during training. For scoring we rely mainly on precision–recall area under the curve.

Overfitting. We controlled overfitting mainly through C. We selected the best C using the validation precision–recall score, then retrained on train plus validation and evaluated once on the test set. We also ran 5-fold StratifiedKFold cross-validation on train plus validation to get a steadier estimate.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 6: Linear SVM





Metric	Mean	Std
PR AUC	0.221918	0.034482
Balanced Accuracy	0.685662	0.012821
Macro F1	0.530131	0.005562

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 7: Histogram Gradient Boost

Rationale. We use Histogram Gradient Boosting because it is usually strong on tabular data and can learn non-linear patterns and feature interactions.

Advantages. It often gives strong results without heavy feature engineering, and it can handle complex relationships better than simple linear models.

Limitations. It can still overfit if the trees grow too complex, and it is less interpretable than simpler baselines.

Hyperparameter. We tune a small grid and choose the best setting by validation PR AUC:

learning_rate: [0.05, 0.1]

max_depth: [None, 6]

max_leaf_nodes: [31, 63]

min_samples_leaf: [20, 50]

l2_regularization: [0.0, 0.1]

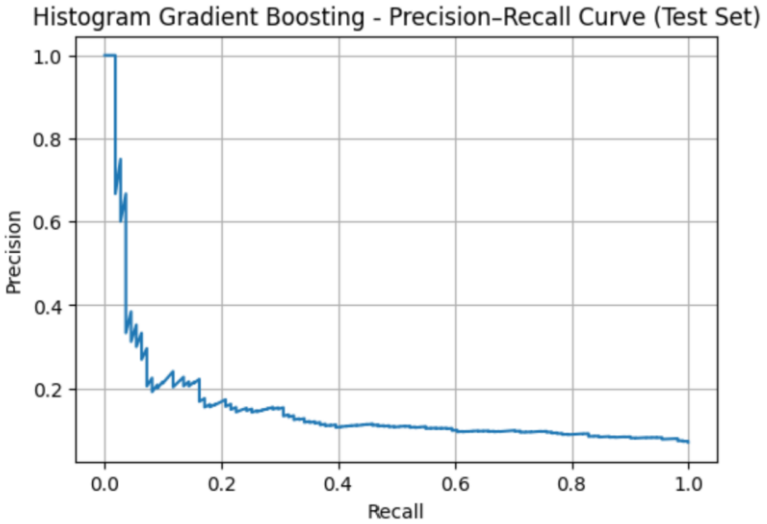
Fixed settings: max_iter = 300,

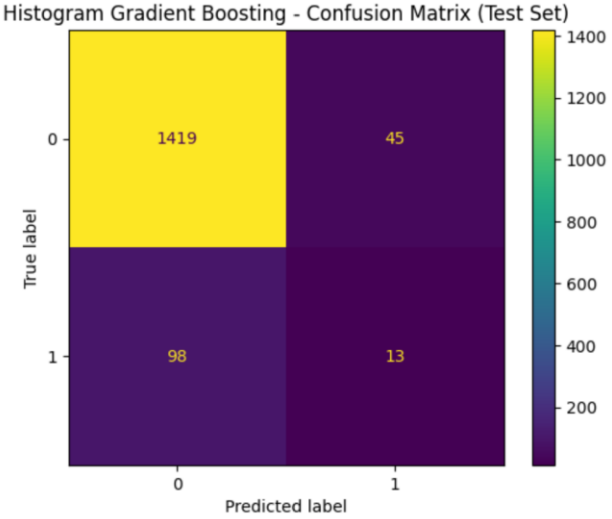
Imbalance. We handle class imbalance using sample weights from compute_sample_weight(class_weight="balanced", y_train) (and the same idea inside each cross-validation fold).

Overfitting. We limit model complexity mainly with max_depth, max_leaf_nodes, min_samples_leaf, and l2_regularization, then we check generalisation using stratified validation and 5-fold stratified cross-validation before reporting final test set results.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 7: Histogram Gradient Boost





Metric	Mean	Std
PR AUC	0.191064	0.040129
Balanced Accuracy	0.542725	0.016282
Macro F1	0.554611	0.021606

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 8: XGBoost

Rationale. We use XGBoost because it is usually one of the strongest choices for tabular data and it can learn complex non-linear patterns and feature interactions.

Advantages. It often performs well with mixed feature types, it captures non-linear relationships, and it is flexible through tuning.

Limitations. It can overfit if the model is too complex, and it needs careful tuning to balance performance and generalisation.

Hyperparameter. We tuned a small grid:

n_estimators = 400, 800; learning_rate = 0.05, 0.1; max_depth = 3, 5;

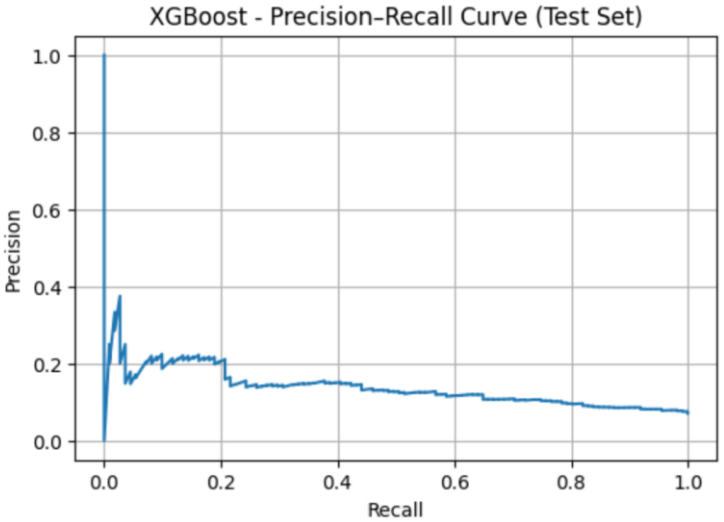
subsample = 0.8; colsample_bytree = 0.8, 1.0; min_child_weight = 1, 5.

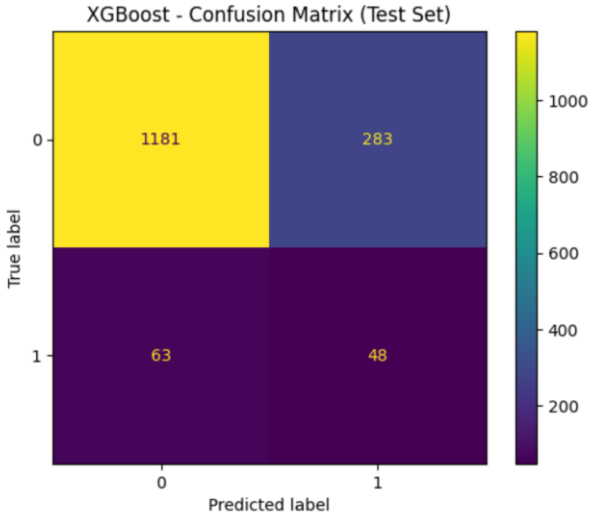
Imbalance. We handled class imbalance using scale_pos_weight, calculated as the ratio of negative to positive samples in the training data, and used the same idea again when training on train plus validation.

Overfitting. We reduced overfitting by limiting model complexity with max_depth and min_child_weight, using subsample and colsample_bytree for extra regularization.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 8: XGBoost





Metric	Mean	Std
PR AUC	0.198011	0.014241
Balanced Accuracy	0.661698	0.013504
Macro F1	0.579506	0.007847

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 9: LihgtGBM

Rationale. We use LightGBM because it is usually a strong choice for tabular data and it can learn non-linear patterns and feature interactions.

Advantages. It often performs well on structured credit style data, handles complex relationships, and does not need feature scaling.

Limitations. It is less interpretable than simple baselines, and it can overfit if the trees are too complex or if boosting runs too long.

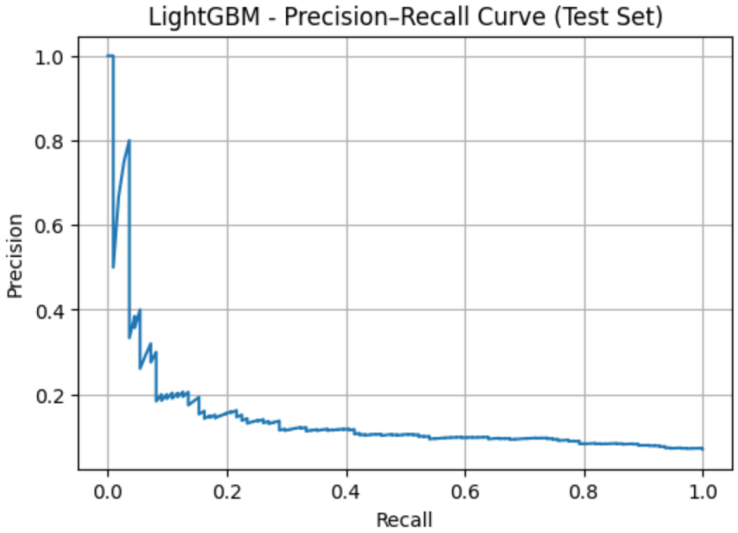
Hyperparameter. We tuned a small grid and selected the best setting: learning_rate = [0.05, 0.1], n_estimators = [400, 800], max_depth = [-1, 8], num_leaves = [31, 63], min_child_samples = [20, 50]. We kept subsample = 0.8, colsample_bytree = 0.8, reg_lambda = 1.0.

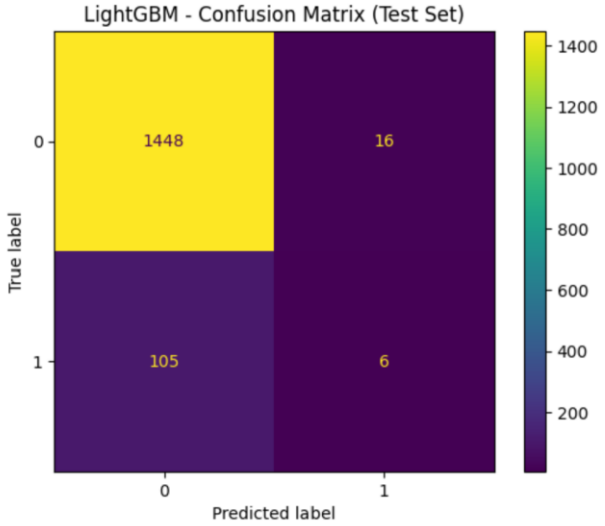
Imbalance. We handled class imbalance using scale_pos_weight, computed from the training split as the ratio of non-default to default cases, and recalculated it again when fitting on train plus validation.

Overfitting. We reduced overfitting by limiting max_depth and num_leaves, requiring more samples in each leaf with min_child_samples, and using subsample and colsample_bytree plus L2 regularisation.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 9: LihgtGBM





Metric	Mean	Std
PR AUC	0.203915	0.039115
Balanced Accuracy	0.541196	0.013433
Macro F1	0.556152	0.020862

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 10: CatBoost

Rationale. We use CatBoost as another strong boosting model for tabular data. It is good at learning non-linear patterns and feature interactions, which can matter for default prediction.

Advantages. It often performs strongly on structured data, handles complex relationships, and usually works well without heavy feature engineering.

Limitations. It is less interpretable than simple models, can be slower to train, and it can overfit if we allow trees to be too complex.

Hyperparameter. We tuned a small grid and chose the best setting:

learning_rate: [0.03, 0.1]

depth: [4, 6, 8]

iterations: [500, 1000]

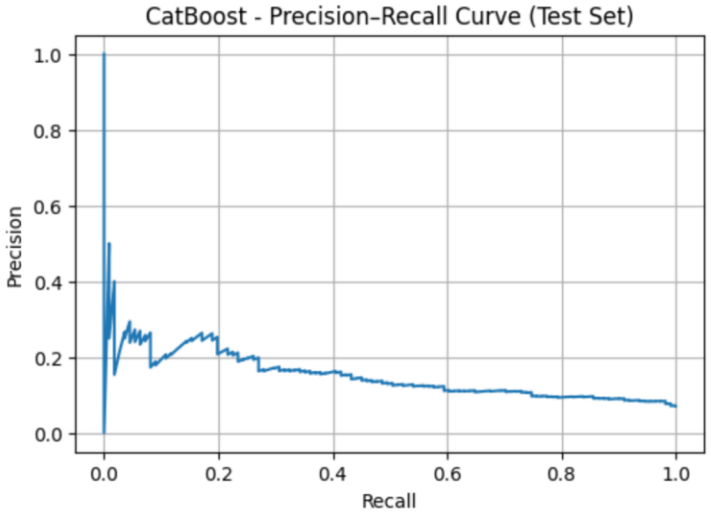
l2_leaf_reg: [1.0, 3.0, 10.0]

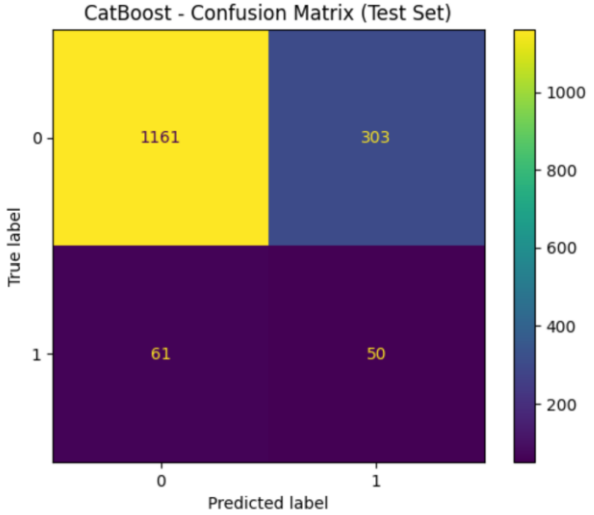
Imbalance. We handled class imbalance using class weights based on the training split, set as [1.0, neg_count / pos_count]. We recomputed the same weights again when training the final model on train plus validation.

Overfitting. We reduced overfitting by limiting model complexity through depth, iterations, and l2_leaf_reg.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 10: CatBoost





Metric	Mean	Std
PR AUC	0.209338	0.021271
Balanced Accuracy	0.668478	0.015139
Macro F1	0.579426	0.008379

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 11: MLP

Rationale. We use a simple deep learning MLP as our first neural network model for this task. It can learn non-linear patterns in tabular data and gives us a clear deep learning baseline to compare against the traditional models.

Advantages. It can capture complex relationships between features, it works well with many input columns, and it outputs probabilities which we evaluate using precision–recall area under the curve.

Limitations. It needs careful tuning and regularisation, it is less interpretable than simple models, and it can overfit if we train too long or use a network that is too large.

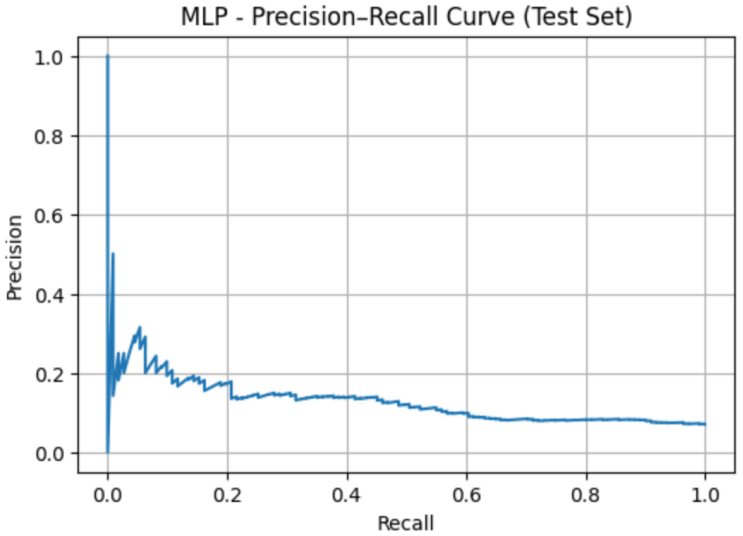
Hyperparameter. We tuned: hidden layer sizes: (128, 64) and (64, 32), dropout: 0.0 and 0.2, learning rate: 1e-3 and 3e-4, weight decay: 0.0 and 1e-4, We used batch_size = 256 and selected the best setup using validation PR AUC. For the final run we trained with early stopping using max_epochs = 60 and patience = 8.

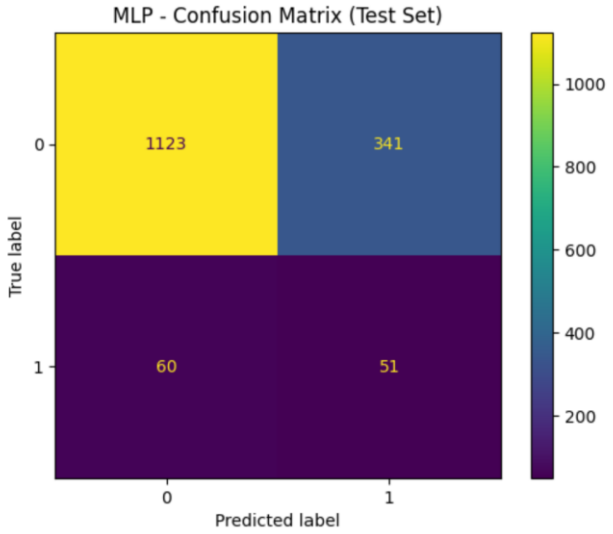
Imbalance. We handled class imbalance inside the loss by using BCEWithLogitsLoss with pos_weight = negatives / positives, so default cases have higher weight during training.

Overfitting. We reduced overfitting using dropout, weight decay with AdamW, and early stopping based on validation PR AUC.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 11: MLP





Metric	Mean	Std
PR AUC	0.223033	0.030050
Balanced Accuracy	0.670831	0.019015
Macro F1	0.556531	0.026538

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 12: MLP with Dropout

Rationale. We use this neural network to capture non-linear patterns in the data. Batch normalization helps training stay stable, and dropout helps generalization.

Advantages. It can model complex relationships between features, and it produces probabilities that work well with precision–recall evaluation on an imbalanced target.

Limitations. It needs careful tuning and regularisation, it is less interpretable than classical models, and it can overfit if training runs too long.

Hyperparameter. We tuned a small grid using the validation PR AUC:

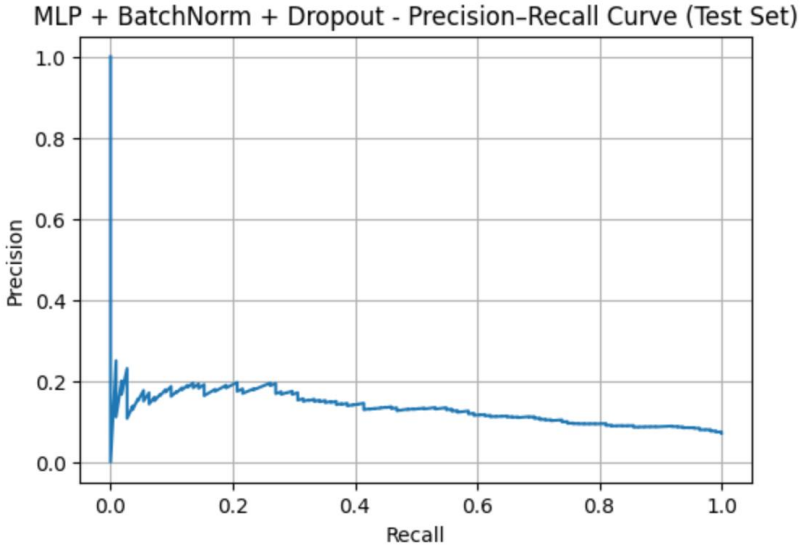
hidden_dims: (256, 128) and (128, 64), dropout: 0.2 and 0.4, learning rate: 0.001 and 0.0003, weight decay: 0.0 and 0.0001, We used batch_size = 256, max_epochs = 40, and patience = 6.

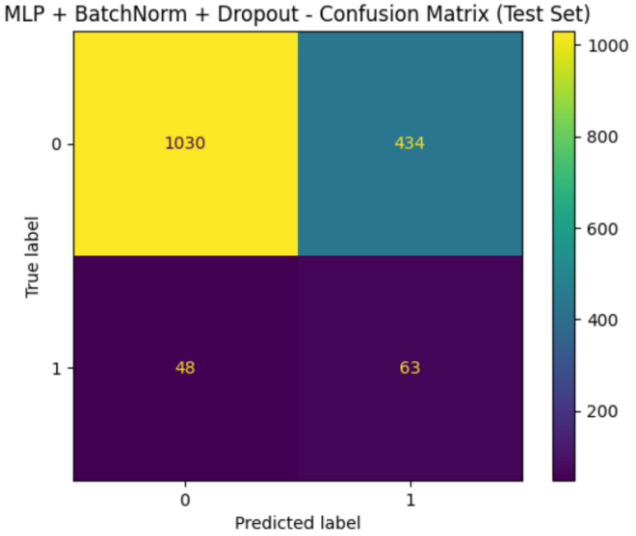
Imbalance. We handled class imbalance inside the loss using BCEWithLogitsLoss with pos_weight calculated as negatives divided by positives in the training split.

Overfitting. We reduced overfitting using dropout, weight decay with AdamW, and early stopping based on validation PR AUC. We also checked stability with 5-fold stratified cross-validation on train plus validation (using max_epochs = 35 and patience = 5 inside each fold).

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 12: MLP with Dropout





Metric	Mean	Std
PR AUC	0.223258	0.038381
Balanced Accuracy	0.678035	0.010005
Macro F1	0.542768	0.005793

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 13: Residual MLP

Rationale. We use a Residual MLP to handle non-linear patterns in tabular data. The skip connections help training stay stable as the network gets deeper.

Advantages. It can learn complex feature interactions, and the residual blocks help gradients flow better than a plain MLP. It outputs probabilities, so it fits our precision–recall based evaluation.

Limitations. It is less interpretable than traditional models and can still overfit if it is too flexible. It also needs more tuning and training time than most classic baselines.

Hyperparameter. We tune:

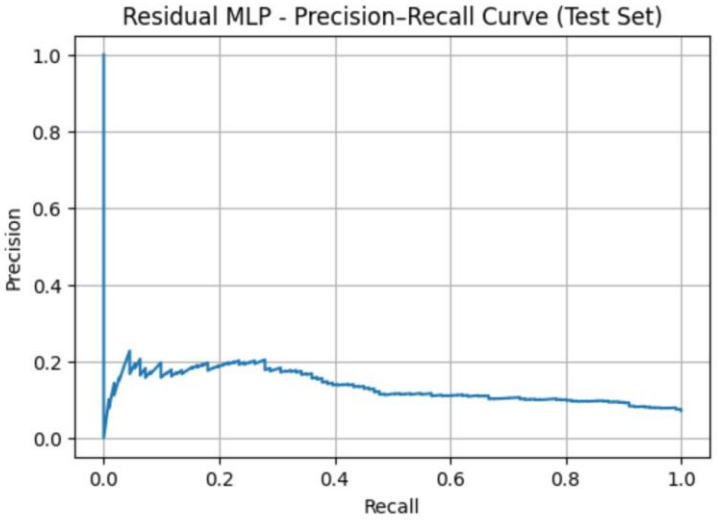
hidden_dim: 128, 256, num_blocks: 2, 3, dropout: 0.2, 0.4, lr: 1e-3, 3e-4, weight_decay: 0.0, 1e-4, Fixed settings: batch_size = 256, max_epochs = 35, patience = 6.

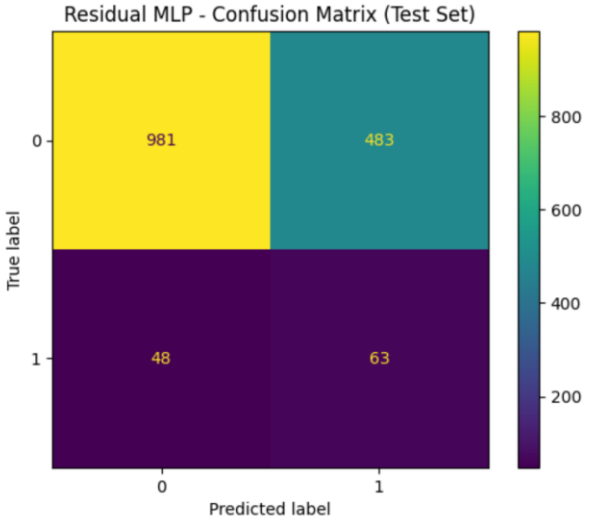
Imbalance. We handle class imbalance using BCEWithLogitsLoss with pos_weight = negatives / positives computed from the training split, so default cases have higher weight during training.

Overfitting. We reduce overfitting using dropout, weight decay with AdamW, and early stopping based on validation PR AUC.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 13: Residual MLP





Metric	Mean	Std
PR AUC	0.218562	0.021930
Balanced Accuracy	0.671059	0.024331
Macro F1	0.491813	0.025328

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 14: TabNet

Rationale. We used TabNet as a deep learning model designed for tabular data. It can learn non-linear patterns and feature interactions, and it also has built-in feature selection behaviour through its attention masks.

Advantages. It can model complex relationships without manual interaction features, and it gives probability outputs that suit our precision–recall evaluation on an imbalanced target.

Limitations. It is heavier to run than classic models, needs extra libraries, and it can be sensitive to training settings. With limited tuning, performance may not reach its full potential.

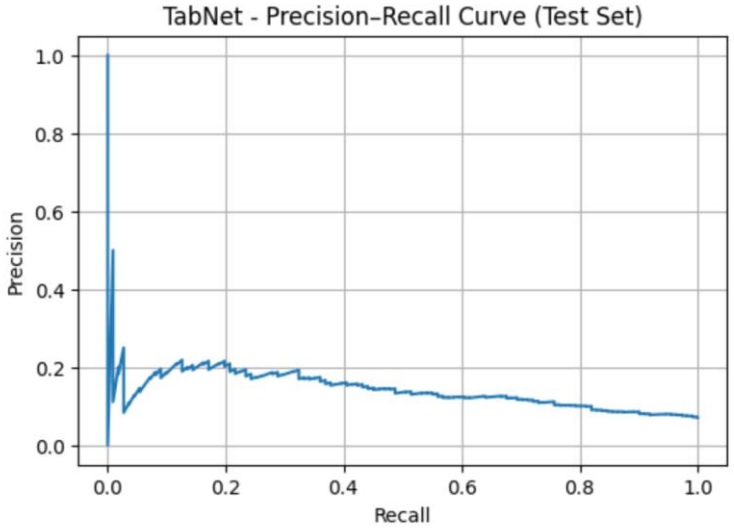
Hyperparameter. Because TabNet is expensive to train, we used a very small tuning grid in the notebook: n_d = 16, n_a = 16, n_steps = 3, gamma = 1.3, lr = 0.02, lambda_sparse = 1e-4. Fixed training settings: mask_type = sparsemax, optimizer = Adam, batch_size = 2048, virtual_batch_size = 256.

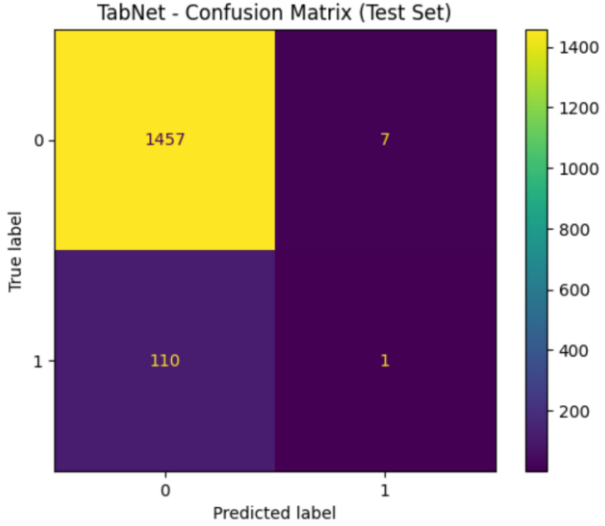
Imbalance. We handled class imbalance with class weights computed from the training split and passed into TabNet as weights. This increases the importance of the default class during training.

Overfitting. We controlled overfitting mainly with early stopping and model size limits. In the final fit we used max_epochs = 60 and patience = 8. We also used the TabNet regularisation settings such as lambda_sparse and kept the network relatively small with n_d, n_a, and n_steps.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 14: TabNet





Metric	Mean	Std
PR AUC	0.135065	0.033169
Balanced Accuracy	0.524019	0.020569
Macro F1	0.525392	0.027718

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 15: FT-Transformer

Rationale. We used FT-Transformer because it is designed for tabular data and can learn complex feature interactions, which can matter when default behavior is difficult to predict. It is also a relatively new deep learning approach based on the Transformer idea, which is the same general family of models used in modern language models such as ChatGPT, but applied here to structured tabular features.

Advantages. It can model non-linear patterns and interactions, and it outputs probabilities which suits our precision–recall evaluation on an imbalanced target. It also works well with a wide feature set.

Limitations. It is heavier to train than classical models, usually needs a GPU or TPU to run efficiently, and it can be sensitive to training settings.

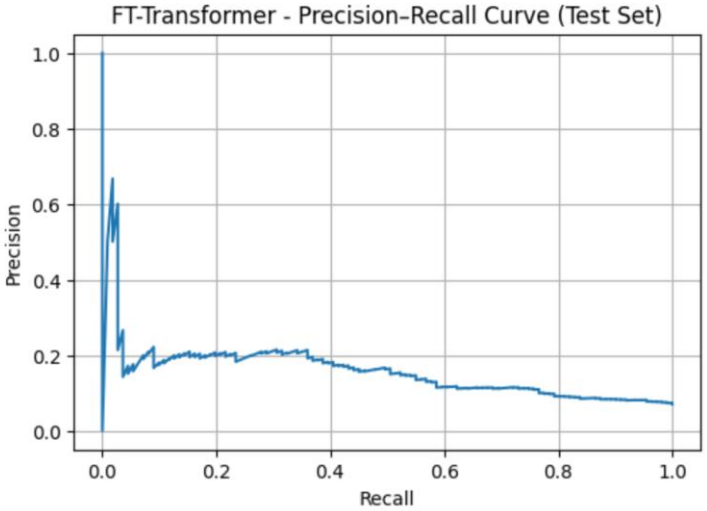
Hyperparameter. We kept tuning minimal because training is expensive. We used this configuration: d_token = 32, n_layers = 2, n_heads = 4, dropout = 0.10, lr = 1e-3, weight_decay = 1e-4, with batch_size = 512, max_epochs = 20, patience = 3, and AdamW as the optimiser.

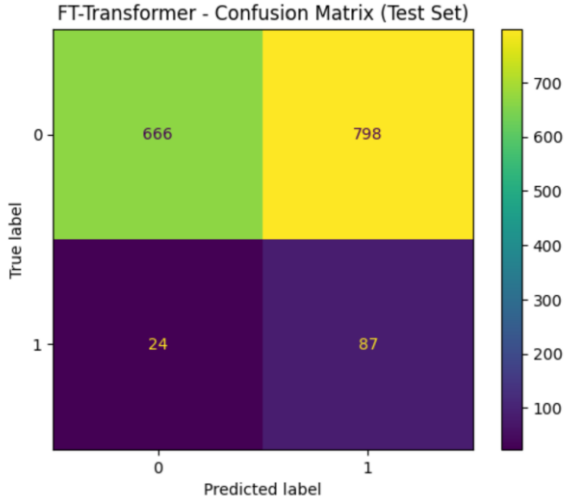
Imbalance. We handled class imbalance using BCEWithLogitsLoss with pos_weight computed from the training split as negatives divided by positives, so the default class has higher weight during training.

Overfitting. We controlled overfitting using dropout, weight decay, and early stopping based on validation precision–recall performance. For the final model we trained on train plus validation, kept a small internal 10 percent split for early stopping, and we also reported a 3-fold stratified cross-validation check because full cross-validation is expensive for this model.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Model 15: FT-Transformer





Metric	Mean	Std
PR AUC	0.194310	0.008409
Balanced Accuracy	0.621532	0.049784
Macro F1	0.452639	0.173782

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

3. Evaluation

Most relevant performance metric

We treat this as an imbalanced classification problem, so we do not rely on plain accuracy.

Instead we report:

PR AUC (Average Precision) as the main metric. It focuses on how well we rank and detect the rare default class, and it is much more informative than accuracy when positives are scarce.

Balanced Accuracy to measure performance fairly across both classes. It avoids the majority class dominating the score.

Macro F1 to balance precision and recall while giving equal importance to both classes, which is useful when the minority class matters most.

Confusion Matrix (plus the precision–recall curve) to clearly show the types of errors, and to support threshold decisions depending on whether we want fewer missed defaults or fewer false alarms.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

3. Evaluation

Robust evaluation technique

We use **5-fold stratified cross-validation** on the combined train and validation data for each model. We report a small summary table with the **mean** and **standard deviation** for **PR AUC (average precision)**, **balanced accuracy**, and **macro F1**.

What this tells us:

The **mean** score is a more reliable estimate than a single validation split, because every row gets used for validation once.

The **standard deviation** shows stability. Small spread means the model performs consistently, large spread means results depend a lot on which fold it sees.

It helps us spot **overfitting**. If training looks strong but cross-validation drops, the model is not generalizing well.

It makes model comparison fairer, especially with an imbalanced target where one split can be misleading.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

3. Evaluation

Results presentation

We present results in three clear ways:

Confusion matrices (test set): for each model we show a confusion matrix so we can see false negatives and false positives. This is important because missing a default and wrongly flagging a good customer have different costs.

Precision–recall curves (test set): we plot the precision–recall curve because our main metric is PR AUC. The curve shows the trade-off between catching more defaults (recall) and keeping predictions reliable (precision) as we change the threshold.

Main results tables: we keep one combined comparison table for all models, showing the key metrics (PR AUC as the main one, plus balanced accuracy and macro F1). This makes it easy to rank models and pick the final candidate.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Leaderboard table

	model	primary_metric	cv_pr_auc_mean	cv_pr_auc_std	cv_balanced_accuracy_mean	cv_balanced_accuracy_std	cv_f1_macro_mean	cv_f1_macro_std
0	MLP + BatchNorm + Dropout	PR AUC (Average Precision)	0.223258	0.038381	0.678035	0.010005	0.542768	0.005793
1	MLP	PR AUC (Average Precision)	0.223033	0.030050	0.670831	0.019015	0.556531	0.026538
2	Linear SVM	PR AUC (Average Precision)	0.221918	0.034482	0.685662	0.012821	0.530131	0.005562
3	Logistic Regression	PR AUC (Average Precision)	0.220787	0.034159	0.684099	0.011757	0.530194	0.005931
4	Residual MLP	PR AUC (Average Precision)	0.218562	0.021930	0.671059	0.024331	0.491813	0.025328
5	Random Forest	PR AUC (Average Precision)	0.212925	0.025418	0.587197	0.014431	0.595913	0.014826
6	CatBoost	PR AUC (Average Precision)	0.209338	0.021271	0.668478	0.015139	0.579426	0.008379
7	LightGBM	PR AUC (Average Precision)	0.203915	0.039115	0.541196	0.013433	0.556152	0.020862
8	XGBoost	PR AUC (Average Precision)	0.198011	0.014241	0.661698	0.013504	0.579506	0.007847
9	FT-Transformer	PR AUC (Average Precision)	0.194310	0.008409	0.621532	0.049784	0.452639	0.173782
10	Histogram Gradient Boosting	PR AUC (Average Precision)	0.191064	0.040129	0.542725	0.016282	0.554611	0.021606
11	Decision Tree	PR AUC (Average Precision)	0.137858	0.008787	0.667937	0.010841	0.457918	0.015391
12	TabNet	PR AUC (Average Precision)	0.135065	0.033169	0.524019	0.020569	0.525392	0.027718
13	Naive Bayes	PR AUC (Average Precision)	0.124086	0.009220	0.531463	0.021165	0.173803	0.044846
14	k-Nearest Neighbours	PR AUC (Average Precision)	0.099428	0.010298	0.517272	0.011844	0.517582	0.018808

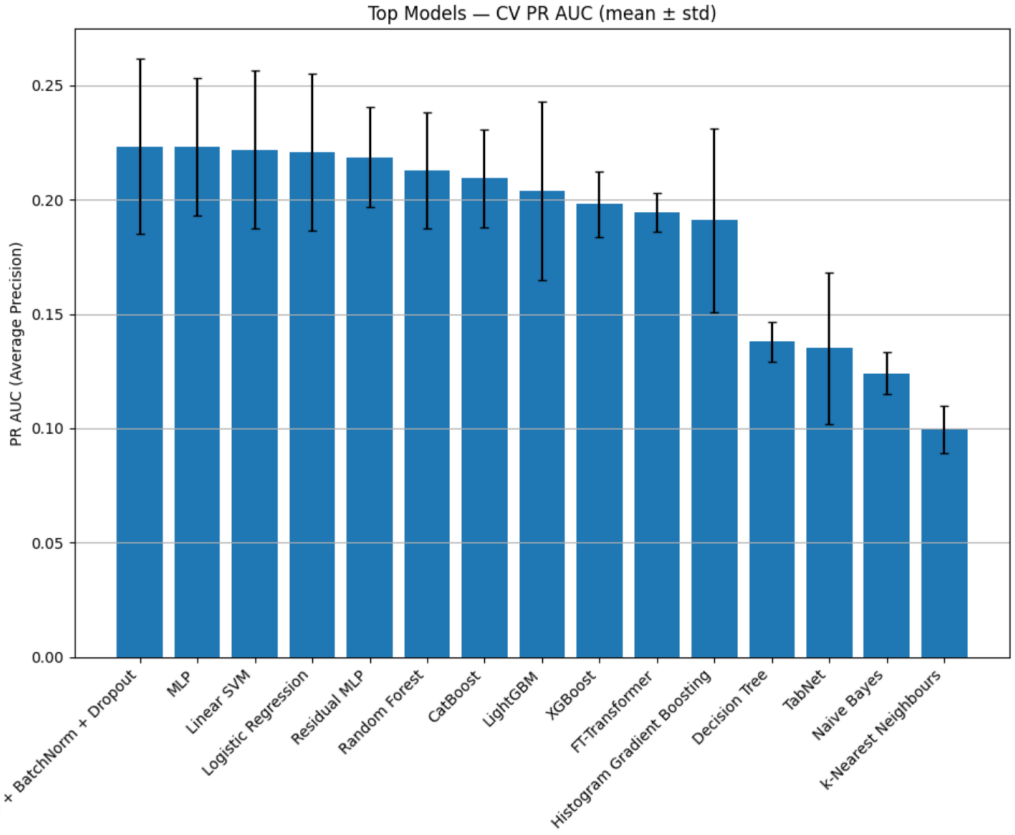
Top models table

Top 15 models by CV PR AUC:

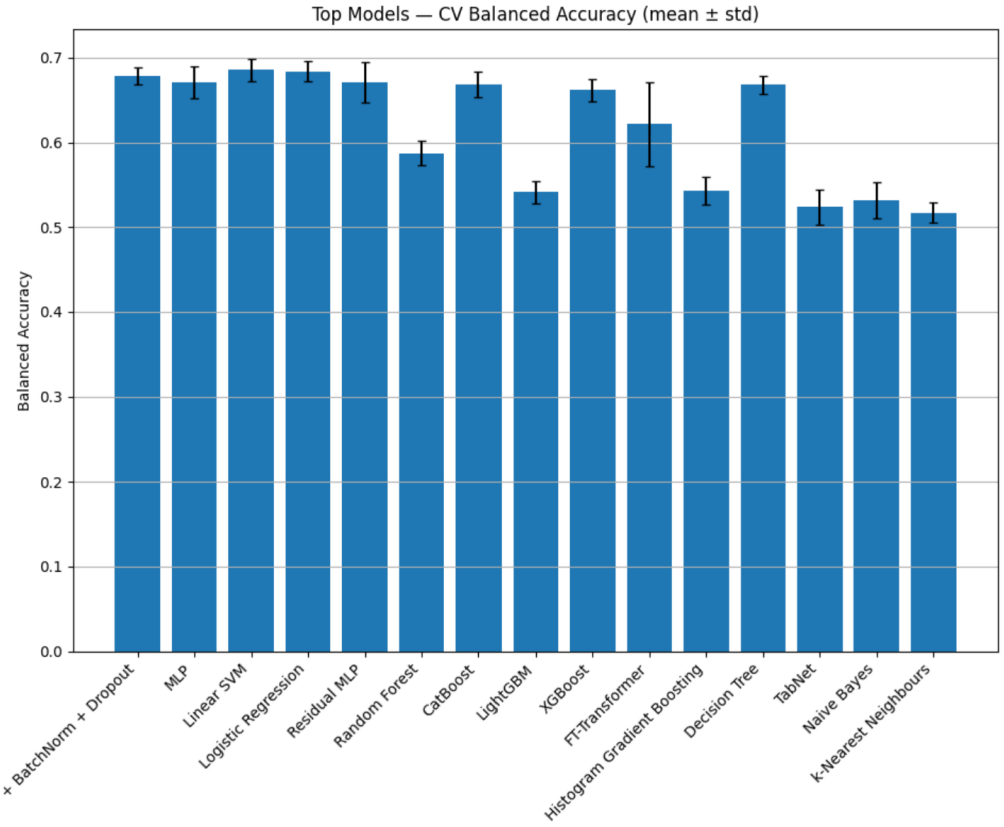
	model	cv_pr_auc_mean	cv_pr_auc_std	cv_balanced_accuracy_mean	cv_balanced_accuracy_std	cv_f1_macro_mean	cv_f1_macro_std
0	MLP + BatchNorm + Dropout	0.223258	0.038381	0.678035	0.010005	0.542768	0.005793
1	MLP	0.223033	0.030050	0.670831	0.019015	0.556531	0.026538
2	Linear SVM	0.221918	0.034482	0.685662	0.012821	0.530131	0.005562
3	Logistic Regression	0.220787	0.034159	0.684099	0.011757	0.530194	0.005931
4	Residual MLP	0.218562	0.021930	0.671059	0.024331	0.491813	0.025328
5	Random Forest	0.212925	0.025418	0.587197	0.014431	0.595913	0.014826
6	CatBoost	0.209338	0.021271	0.668478	0.015139	0.579426	0.008379
7	LightGBM	0.203915	0.039115	0.541196	0.013433	0.556152	0.020862
8	XGBoost	0.198011	0.014241	0.661698	0.013504	0.579506	0.007847
9	FT-Transformer	0.194310	0.008409	0.621532	0.049784	0.452639	0.173782
10	Histogram Gradient Boosting	0.191064	0.040129	0.542725	0.016282	0.554611	0.021606
11	Decision Tree	0.137858	0.008787	0.667937	0.010841	0.457918	0.015391
12	TabNet	0.135065	0.033169	0.524019	0.020569	0.525392	0.027718
13	Naive Bayes	0.124086	0.009220	0.531463	0.021165	0.173803	0.044846
14	k-Nearest Neighbours	0.099428	0.010298	0.517272	0.011844	0.517582	0.018808

Saved: model_leaderboard.csv

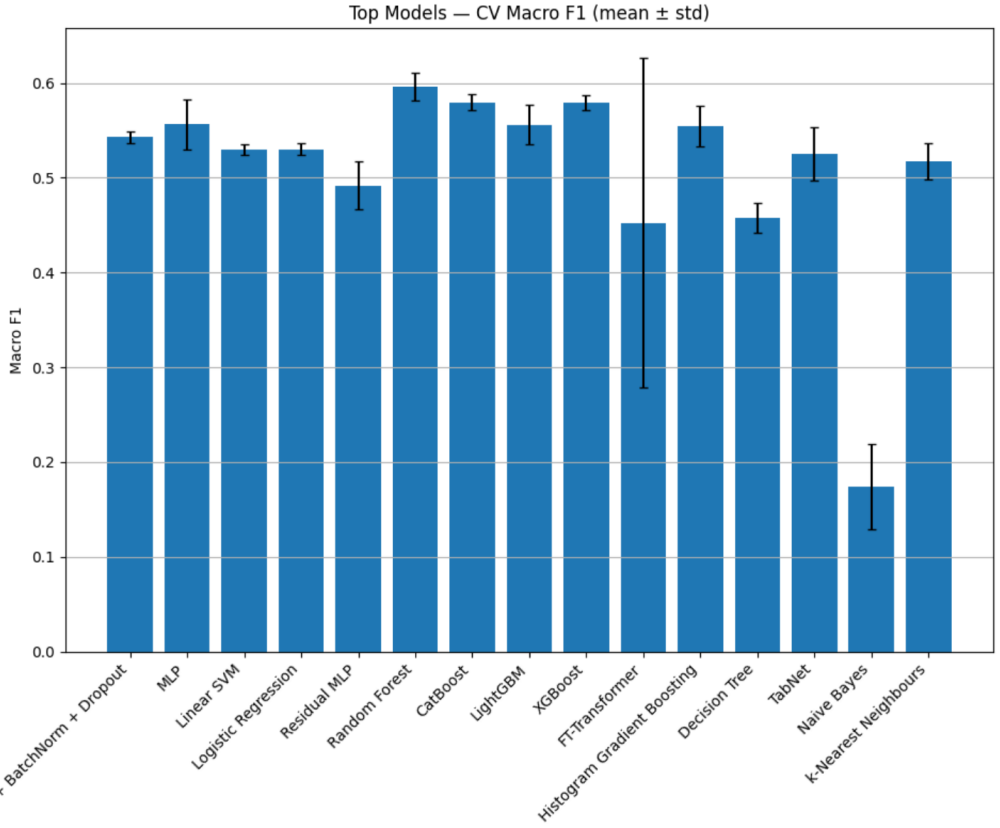
Visual comparision CV PR AUC



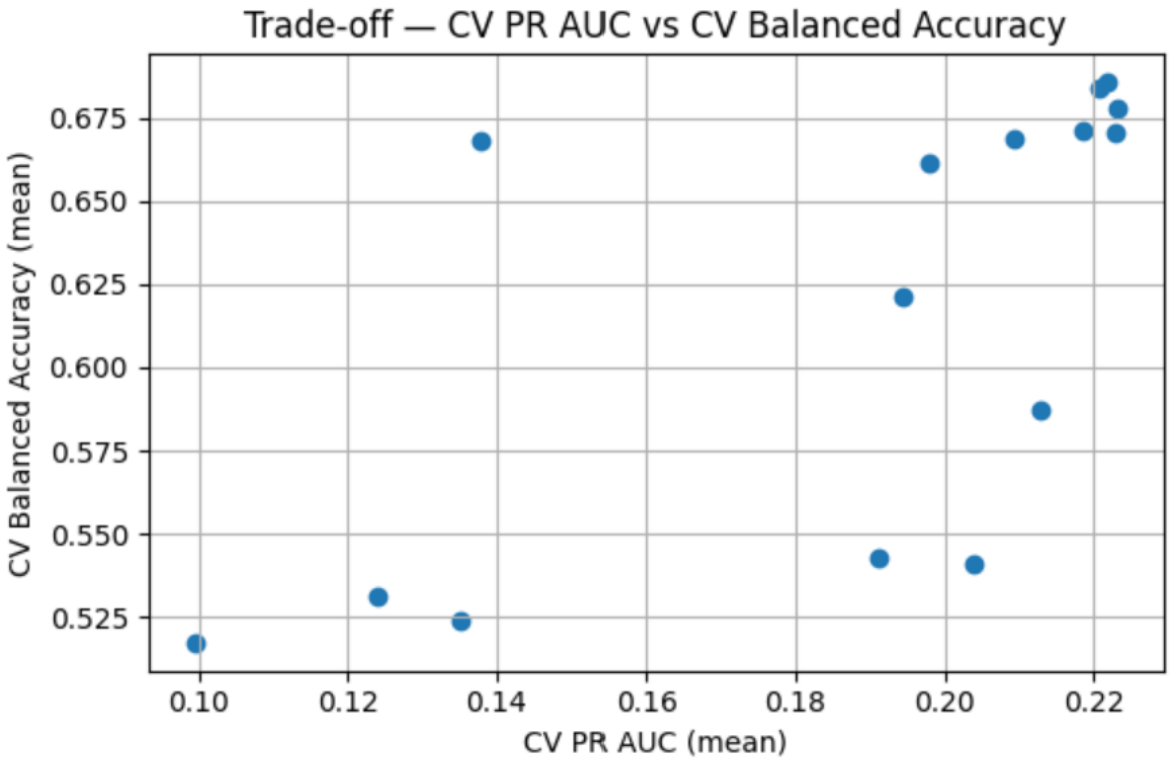
Visual comparision CV Balanced Accuracy



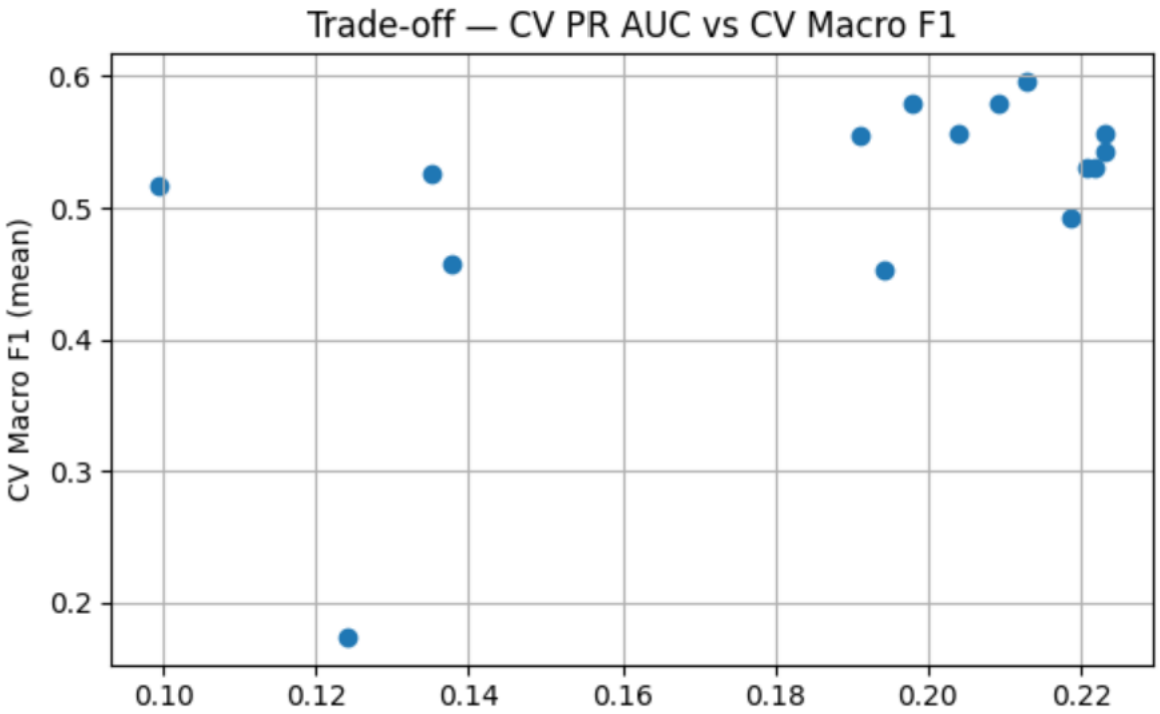
Visual comparision CV Macro F1



Visual comparision CV PR AUC vs CV Balanced Accuracy



Visual comparision CV PR AUC vs CV Macro F1



Insight: Most stable model (lowest CV PR AUC std)

Most stable (lowest CV PR AUC std):

	model	cv_pr_auc_mean	cv_pr_auc_std
9	FT-Transformer	0.194310	0.008409
11	Decision Tree	0.137858	0.008787
13	Naive Bayes	0.124086	0.009220
14	k-Nearest Neighbours	0.099428	0.010298
8	XGBoost	0.198011	0.014241

Insight: Biggest CV and test drop (possible overfit)

```
Largest CV → test PR AUC drop:
```

	model	cv_pr_auc_mean	test_pr_auc	cv_to_test_pr_auc_drop
1	MLP	0.223033	0.131191	0.091843
0	MLP + BatchNorm + Dropout	0.223258	0.134881	0.088377
4	Residual MLP	0.218562	0.134843	0.083719
2	Linear SVM	0.221918	0.151649	0.070269
3	Logistic Regression	0.220787	0.150866	0.069921

Insight: Multi-metric score (we just sum 3 metrics)

```
Best overall by simple rank-sum (PR AUC + Balanced Accuracy + Macro F1):
```

	model	cv_pr_auc_mean	cv_balanced_accuracy_mean	cv_f1_macro_mean	rank_sum
0	MLP + BatchNorm + Dropout	0.223258	0.678035	0.542768	11.0
1	MLP	0.223033	0.670831	0.556531	11.0
2	Linear SVM	0.221918	0.685662	0.530131	13.0
3	Logistic Regression	0.220787	0.684099	0.530194	14.0
4	CatBoost	0.209338	0.668478	0.579426	16.0
5	Random Forest	0.212925	0.587197	0.595913	17.0

Final choice

```
Best CV PR AUC: MLP + BatchNorm + Dropout 0.2325765436883255
Best CV Balanced Accuracy: Linear SVM 0.6856618430797803
Best CV Macro F1: Random Forest 0.5959125772808445
Most stable (lowest CV PR AUC std): FT-Transformer 0.008409247481279292
```

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Limitations and challenges

We summarize the main issues we saw in our project:

- The target is heavily imbalanced, so defaults are rare.
- Because of this, accuracy can look high even when the model is not catching many defaults.
- A fixed 0.5 probability threshold is usually not the best choice for an imbalanced target.
- Sometimes cross-validation and test results do not match. This can point to overfitting or a small shift in the data distribution.
- Data issues like missing values, outliers, heavy-tailed money features, and our encoding choices can change model behavior.
- Deep learning models and transformer-based models take longer to train and they are harder to tune, so we kept tuning limited.
- Using a GPU or TPU helps reduce training time, which means we can afford more experiments and more careful hyperparameter tuning.

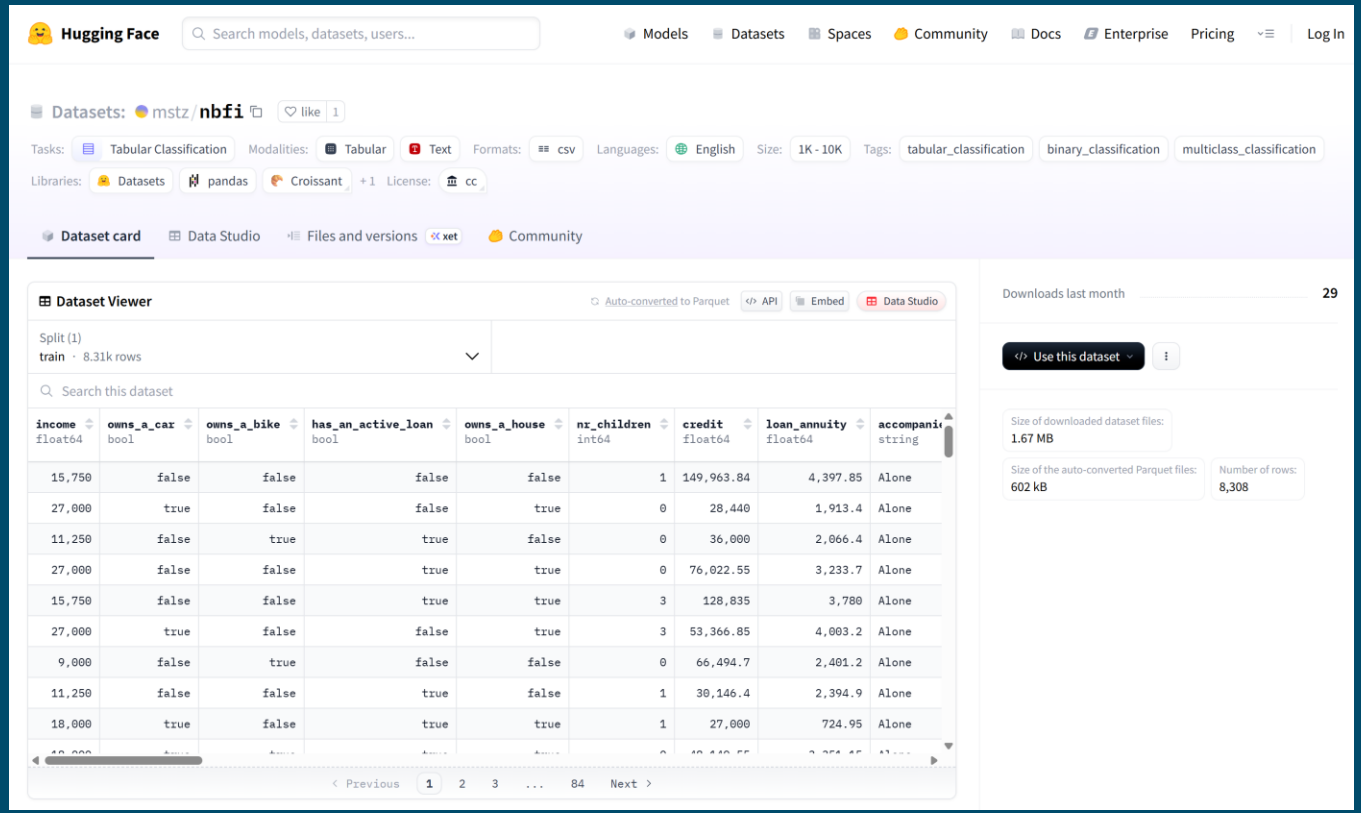
OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Future work and model improvements

We list a few things we can try next to improve the results :

- Do better hyperparameter search, but only for the best models.
- Use automatic feature selection or dimensionality reduction to test which features really help and which ones add noise.
- Try an ensemble, for example combine the best boosting model with the best deep learning model.
- Optimize training for GPU.
- Optimize training for TPU. TPU can calculate large matrices very fast, so we can train deep models quicker, try more tuning, and models like FT-Transformer may give better performance with enough training.
- Keep searching for new models and new methods.

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN



Click on image to open dataset

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TUDUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN